

Markdown Studio 유지보수 안내

두 부분으로 이뤄집니다.

- **웹 앱** `markdown-studio.html` — 단일 자체완결형 HTML(빌드·의존성 없음).
- **데스크톱 래퍼** `desktop/` — 위 HTML을 `app://` 특권 스킴으로 서빙하는 Electron 앱 + 패키징.

이 문서는 **코드를 고칠 때 지켜야 할 보안 불변식, 자주 읽는 함정, 회귀 테스트, 데스크톱 빌드-배포**를 정리합니다. 보안 이력은 `[[SECURITY-REVIEW-2026-08-04]]`.

1. 절대 어기면 안 되는 보안 불변식

(1) 파서가 유일한 신뢰 경계다

`render()` + `inline()` 는 **미리보기와 모든 내보내기(HTML/PDF/DOCX)의 단일 진실 공급원**입니다. 두 번째 파서를 만들지 마십시오. 렌더 결과를 다루는 새 기능은 반드시 이 파이프라인을 거쳐야 합니다.

(2) 문서에서 유래한 HTML·URL은 반드시 살균한다

`.md` 내용은 **신뢰 불가 입력**입니다. 문서 본문의 원시 HTML·링크·이미지 URL을 화면·산출물에 넣기 전 아래를 통과시키십시오.

대상	함수
인라인 원시 HTML 태그(단일 태그)	<code>sanitizeInlineTag()</code>
블록 원시 HTML 조각(여러 줄)	<code>sanitizeFragment()</code>
링크·이미지 URL	<code>isSafeUrl(url, allowDataImage)</code>

반대로 **앱이 스스로 조립하는 마크업**(코드블록 툴바의 `<select>` · `<button>` · `<svg>` , 프론트매터 카드 등)은 신뢰 대상이라 살균하지 않습니다. 살균기를 렌더 결과 전체에 걸면 이 UI가 깨지니 절대 금지.

(3) innerHTML 에 넣는 문서 유래 문자열은 esc() 또는 살균을 거친다

속성값은 esc() 후 큰따옴표로 감싸고, 원시 HTML은 위 (2)의 살균기, 본문 텍스트는 esc() .

(4) 새 링크엔 rel="noopener noreferrer" , 새 URL엔 스킴 검사

target="_blank" 링크엔 반드시 rel. href / src 엔 반드시 isSafeUrl() .

(5) CSP를 무력화하지 마라

'unsafe-eval' 을 추가하거나(=eval 도입) 인라인 이벤트 핸들러(onclick= 등)를 앱 HTML에 직접 넣지 마십시오. CSP 유지 조건이 깨집니다(→ §3).

(6) 데스크톱: 특권 창을 외부로 내보내지 마라

app:// 창은 특권(secure context)입니다. webPreferences 의
nodeIntegration:false · contextIsolation:true · sandbox:true , 새 창 거부, will-navigate
차단을 완화하지 마십시오.

2. 살균기 확장 방법

정책은 허용 목록(allowlist) 방식입니다.

```
// 태그 추가

const SAFE_TAGS = new Set((" ... 태그명 추가 ... ").split(" "));

// 속성 추가 - 태그별 또는 전역("*")

const SAFE_ATTR = {
  "*": new Set([ ... 공통 허용 속성 ... ]),
  a: new Set(["href"]), // target.rel 은 코드가 강제로 다시 불임
  img: new Set(["src", "alt", "width", "height"]),
};
```

- attrAllowed() 는 on* (이벤트 핸들러)과 style 을 무조건 차단합니다(허용 목록에 넣어도 통과 안 됨, 이중 안전장치). 인라인 style 을 다시 허용하려면 별도 CSS 속성 화이트리스트를 두고 보안 검토를 거치십시오.

- URL 스킴 정책은 `isSafeUrl()` 한 곳에서 관리:
 - 허용: `http(s)` · `mailto` · `tel` · 상대경로 · #앵커
 - 차단: `javascript:` · `vbscript:` · `file:` · `blob:`
 - `data:` 는 **이미지에서만**, `data:image/` 만, `data:image/svg` 제외
 - `sanitizeInlineTag()` (문자열)와 `sanitizeFragment()` (DOM)는 **같은 정책 집합**을 공유합니다. 정책을 바꾸면 둘 다 반영됩니다. 인라인은 짝이 안 맞는 단일 태그를, 블록은 `<template>` DOM으로 짝 맞는 조각을 처리합니다.
-

3. CSP 유지 규칙

CSP는 두 곳에 있습니다.

1. **앱 본체** — `markdown-studio.html` `<head>` 의 메타 태그. 앱이 인라인 `<script>` / `<style>` 을 쓰므로 그 둘만 `'unsafe-inline'` . **`'unsafe-eval'` 은 넣지 마십시오.** 외부 이미지·내보내기 `fetch` 때문에 `img/connect` 에 `http(s)` `data: blob:` 허용.
2. **내보낸 HTML** — `buildStandaloneHtml()` 의 `<head>` . `default-src 'none'` 계열로 스크립트를 원천 차단. 내보낸 문서에 스크립트를 추가하지 마십시오.

주의: 새 외부 리소스 유형(폰트 CDN, 웹워커, `iframe` 임베드 등)을 도입하면 해당 CSP 지시어 (`font-src` / `worker-src` / `frame-src` 등)를 함께 갱신해야 기능이 깨지지 않습니다. 수정 후 §5의 CSP 회귀를 반드시 확인하십시오.

4. 파일 편집 시 함정 (중요)

이 파일에는 **리터럴 제어문자**가 있어 편집 도구가 오작동하기 쉽습니다.

(A) U+0000 / U+0001 자리표시자

`inline()` 은 코드·원시 HTML을 임시로 가릴 때 **실제 제어문자 U+0000·U+0001**을 자리표시자로 씁니다. 이 때문에 `grep` / 검색이 파일을 **바이너리로 인식**하거나, 그 줄 근처에서 문자열 매칭 편집이 실패합니다. → 제어문자를 포함하지 않는 **고유한 인접 텍스트**를 앵커로 편집하십시오.

(B) \uxxxx 이스케이프가 실제 문자로 치환되는 문제 ⚠

일부 편집 도구는 소스에 쓴 `\s` 같은 이스케이프를 실제 제어문자로 바꿔 저장합니다. 실제로 `isSafeUrl` 의 문자 범위를 `-` 로 쓰자 파일에 리터럴 **NUL**이 저장돼 브라우저가 정규식 범위 오류 (Range out of order)를 던졌고 스크립트 전체가 죽었습니다.

- 교훈: 정규식의 제어문자 범위는 `\uxxxx` 대신 `\s` 같은 안전한 메타문자를 쓰거나 PowerShell 등으로 **바이트 단위 치환**을 하십시오. (현재 코드는 `/\s+/g` 로 안전.)
- 편집 후 항상 브라우저 콘솔에 **구문 오류가 없는지** 확인(→ §5-1).

(C) 인코딩

UTF-8(BOM 없음)로 저장하십시오. NUL(0x00)·SOH(0x01)을 포함하므로 텍스트 도구가 이 바이트를 보존하는지 확인(PowerShell: `New-Object System.Text.UTF8Encoding($false)`).

5. 회귀 테스트 (수정 후 반드시 실행)

5-1. 구동·콘솔 점검

```
python -m http.server 5173  
  
# http://localhost:5173/markdown-studio.html 열기  
  
# DevTools 콘솔에 오류(특히 SyntaxError)·CSP 위반(Refused to...)이 없어야 함
```

5-2. XSS 회귀 스니펫 (페이지 콘솔에 붙여넣기)

`executed` 가 **0**, `leaks` 가 빈 배열, `legitPreserved` 가 **true**여야 통과.

```

(() => {
  const fired = []; const real = window.alert; window.alert = m => fired.push(m);
  const P = {
    a: '<img src=x onerror="alert(1)">', b: '<svg onload="alert(2)"></svg>',
    c: '<iframe src="javascript:alert(3)"></iframe>', d: '[x](javascript:alert(4))',
    e: '! [x](javascript:alert(5))', f: '[x](javascript:&#58;alert(6))',
    g: '<a href="#" onclick="alert(7)">x</a>'
  };
  const box = document.createElement('div'); document.body.appendChild(box);
  const leaks = [];
  for (const [k,src] of Object.entries(P)) {
    box.innerHTML = render(src, true);
    box.querySelectorAll('*').forEach(el => {
      el.dispatchEvent(new MouseEvent('click'));
      for (const at of el.attributes) {
        const v = at.value.replace(/\s/g,'').toLowerCase();
        if (at.name.startsWith('on')) leaks.push(k+'on');
        if ((at.name==='href' || at.name==='src') && /^(javascript|vbscript):/.test(v)) leaks.p
      }
    });
  }
  const lh = render('<b>b</b> <kbd>K</kbd> [s](https://ex.com) <a href="https://a.com" target
  const legitPreserved = /<b>b</b>/.test(lh) && /<kbd>K</kbd>/.test(lh)
    && /<a href="https:\/\/ex.com"/.test(lh) && /rel="noopener"/.test(lh);
  box.remove(); window.alert = real;
  console.log(JSON.stringify({ executed: fired.length, leaks, legitPreserved }));
})();

```

5-3. CSP 회귀

페이지 콘솔에서 아래를 실행해 내보내기·PDF 경로가 CSP 위반 없이 동작하는지 확인(모두 true여야 함).

```
(async () => {
  newDoc();
  const b = await buildStandaloneHtml(cur());
  const exportOK = typeof b.html === 'string' && /Content-Security-Policy/.test(b.html) && /d
  const fr = document.createElement('iframe'); fr.style.cssText='position:fixed;width:0;height:0;
  document.body.appendChild(fr); const d = fr.contentWindow.document;
  d.open(); d.write('<!doctype html><style>p{color:rgb(1,2,3)}</style><p id=t>hi</p>'); d.close();
  await new Promise(r=>setTimeout(r,60));
  const iframeOK = d.getElementById('t') && getComputedStyle(d.getElementById('t')).color==='
  fr.remove();
  console.log(JSON.stringify({ exportOK, iframeOK }));
})();
```

그리고 로드 직후 콘솔에 Refused to... CSP 위반이 없어야 합니다.

5-4. 기능 스모크

새 문서 → 제목·목록·코드블록 렌더 / 코드블록 언어 드롭다운 변경 / 다른 이름으로 → HTML·PDF·Word 내보내기(이미지 포함 문서 포함).

5-5. Electron 셀프테스트(데스크톱 수정 시)

```
# 개발 모드: 보안컨텍스트·FSA·렌더·파일오픈까지 헤드리스 검증
$env:MDS_SELFTEST="1"; $env:MDS_SELFTEST_OUT="$env:TEMP\mds.json"
node_modules\electron\dist\electron.exe . "어떤파일.md"
Get-Content $env:MDS_SELFTEST_OUT # 모든 결과 pass, 마지막에 docs=1 [*.md(RO)] 이어야 함
```

6. 데스크톱 빌드·배포

6-1. 빌드 명령

```
cd desktop

npm install          # Electron 런타임 포함(최초 1회)
npm start            # 개발 실행(리포지토리 루트 HTML 로드)
npm test             # 헤드리스 기능 테스트
npm run pack         # 커스텀 아이콘 빌드 → dist/Markdown Studio-win32-x64/ (단, SAC 차단)
npm run pack:portable # ★SAC 통과 빌드 → dist/MarkdownStudio-portable/ (권장)
```

6-2. Smart App Control(SAC) — 반드시 이해할 것

- `npm run pack` 은 원본 `electron.exe` 를 `rcedit`로 재기록하며 **바이트를 바꿔** Microsoft ISG 평판 해시를 없앱니다 → SAC가 "안전하지 않을 수 있는 앱"으로 차단.
- `npm run pack:portable` (`build-portable.ps1`)은 원본 `electron.exe` 를 **바이트 그대로 rename**만 하므로 평판이 유지돼 SAC를 통과합니다. 스크립트가 원본과 SHA-256 해시 일치를 자동 검증합니다.
- 따라서 **코드 수정 후에는 pack 이 아니라 pack:portable 로 재빌드**해야 SAC 통과가 유지됩니다.
- 트레이드오프: 포터블 빌드는 일반 Electron 아이콘/버전 정보. **커스텀 아이콘 유지 + 배포**가 필요하다면 신뢰된 인증서(가능하면 EV)로 코드 서명하십시오(`signtool` 또는 `electron-builder win.certificateFile`).

6-3. 파일 연결(.md 더블클릭)

- `main.js` 의 `fileArgFrom(argv)` 가 실행 인자에서 파일 경로를 찾고, `openFileInWindow()` 가 **읽기전용 뷰어**로 엽니다. 단일 인스턴스 락 + `second-instance` 로 앱이 켜져 있으면 그 창에 전달.
- 연결 대상 `exe`는 `dist\MarkdownStudio-portable\Markdown Studio.exe` (폴더 전체 유지).
- **편집 모드로 열리게** 하려면 `openFileInWindow` 의 `readonly:true` → `false` 후 `npm run pack:portable` .

7. 향후 하드닝 로드맵 (선택, 우선순위 순)

- 1. **코드 서명(EV)** — SAC/SmartScreen 신뢰를 정식으로 확보하고 커스텀 아이콘 유지. 사내 인증서 확인.
- 2. **DOMPurify 전환** — 신뢰 불가 문서를 대량 취급하면 자체 살균기를 DOMPurify로 교체(단일 파일 유지를 위해 소스 인라인 검토).
- 3. **안전한 인라인 style 화이트리스트** — 표현 손실이 문제면 `color · text-align` 등 무해 속성만 허용.
- 4. **내보내기 이미지 fetch 범위 제한(V8)** — 사설/내부망 주소 차단 옵션.

8. 코드 지도 (빠른 참조)

웹 앱 `markdown-studio.html`

기능	핵심 심볼
보안 살균	<code>isSafeUrl</code> , <code>decodeEntities</code> , <code>sanitizeInlineTag</code> , <code>sanitizeFragment</code> , <code>SAFE_TAGS</code> , <code>SAFE_ATTR</code> , <code>attrAllowed</code>
CSP	<code><head></code> 메타(앱 본체), <code>buildStandaloneHtml()</code> 내 <code><head></code> 메타(내보내기)
마크다운 렌더	<code>render(src, top)</code> , <code>inline(s)</code> , <code>esc(s)</code>
코드 하이라이트	HL (자체 IIFE), <code>effLang</code> , <code>langSelect</code>
다중 문서	<code>docs[]</code> , <code>cur()</code> , <code>select()</code> , <code>drawTabs()</code> , <code>paint()</code>
최근 문서	<code>idb</code> , <code>touchRecent</code> , <code>loadRecents</code> , <code>drawRecents</code>
파일 입출력	<code>openDoc</code> , <code>writeTo</code> , <code>saveAs</code> , <code>openFromFolder</code> , <code>handleDrop</code>
내보내기	<code>buildStandaloneHtml</code> (HTML/PDF), <code>buildDocxBlob + zipStore</code> (DOCX), <code>inlineImages</code>
찾기·바꾸기	<code>fbSearch</code> , <code>fbReplaceOne</code> , <code>fbReplaceAll</code>

데스크톱 desktop/

파일	역할
main.js	app:// 스킴 서빙, 창 생성, 파일 연결 오픈 (fileArgFrom · openFileInWindow ·단일 인스턴스), 보안 하드닝 (sandbox ·권한 화이트리스트· will-navigate ·새 창 거부), 셸프테스트
build-portable.ps1	SAC 통과용 서명 보존 포터블 빌드 생성(npm run pack:portable)
package.json	pack / pack:portable / dist 스크립트
README.md	빌드·SAC 대응·파일 연결 안내

세부 아키텍처는 저장소 CLAUDE.md , 보안 이력은 [[SECURITY-REVIEW-2026-08-04]] 참조.